



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

PROYECTO "APLICACIÓN MÓVIL COLABORATIVA PARA MEJORAR LA EFECTIVIDAD EN LA ASISTENCIA DE MASCOTAS PERDIDAS CON PARTICIPACIÓN DE VOLUNTARIOS EN LA CIUDAD DE TACNA, 2026"

Curso: Construcción de software II

Docente: RICARDO EDUARDO VALCARCEL ALVARADO

Integrantes:

<i>Chite Quispe Brian Danilo</i>	<i>(2021070015)</i>
<i>Anampa Pancca David Jordan</i>	<i>(2022074268)</i>

Tacna – Perú
2026

Planificación, ejecución y seguimiento de iteraciones de laboratorio

Control de Versiones

Versión	Fecha	Autor	Descripción
1.0	12/09/2026	Chite Quispe Brian Danilo / Anampa Pancca David Jordan	Elaboración del Plan de Iteración detallado para los 8 sprints del proyecto SOS Mascota, en base a la plantilla corporativa y al Sprint Backlog del curso.

Arquitectura General del Sistema

Antes de detallar cada iteración, se resume la arquitectura común que sustenta a todos los requerimientos, ya que gran parte de las decisiones técnicas descritas en cada sprint (rendimiento, seguridad, tiempo real) dependen de este diseño base:

- Frontend móvil: Flutter (Dart), con gestión de estado basada en Streams/Provider para reflejar en tiempo real los cambios de Firestore.
- Backend as a Service: Firebase Authentication (identidad), Cloud Firestore (base de datos NoSQL en tiempo real), Firebase Storage (imágenes), Firebase Cloud Messaging (notificaciones push) y Cloud Functions (lógica de servidor: triggers, agregaciones y asignación de roles).
- Geolocalización: Google Maps SDK for Flutter y el paquete geolocator para obtener y renderizar coordenadas de reportes y avistamientos.
- Seguridad: control de acceso mediante reglas de seguridad de Firestore basadas en el UID del usuario autenticado y, para funciones administrativas, en Custom Claims del token de Firebase Authentication.

Esta arquitectura explica por qué los requerimientos no son piezas aisladas: casi todos leen o escriben sobre las mismas colecciones ("usuarios", "reportes", "avistamientos"), por lo que una decisión tomada en un sprint temprano (por ejemplo, el esquema de "reportes" en el Sprint 2) condiciona directamente el diseño de los sprints posteriores (listado, moderación, estadísticas).

Link: https://github.com/David-Anampa/SosMascota_Construcci-n02_Anampa_Chite.git

master

1 Branch

0 Tags

Go to file

Add file

Code

David-Anampa

Avance al 100%

373e7a9 · 2 months ago

6 Commits

.firebase	Avance V1	5 months ago
android	Avance 90%	3 months ago
assets	Avance V1	5 months ago
ios	Avance V1	5 months ago
lib	Avance al 100%	2 months ago
linux	Avance V1	5 months ago
macos	Avance V1	5 months ago
test	Avance al 100%	2 months ago
web	Avance V1	5 months ago
windows	Avance V1	5 months ago
.firebasearc	Avance 2	4 months ago
.gitignore	Avance V1	5 months ago
.metadata	Avance V1	5 months ago
README.md	Avance V1	5 months ago
analysis_options.yaml	Avance V1	5 months ago
firebase.json	Avance 10	3 months ago
firestore.indexes.json	Avance 10	3 months ago
firestore.rules	Avance al 99%	3 months ago
pubspec.yaml	Avance V1	5 months ago

About

7-implementacion-de-codigo-g03_chite_anampa-a created by GitHub Classroom

Readme

Activity

Custom properties

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors

David-Anampa David Jordan Anam...

Languages

Dart 92.9% C++ 3.5% CMake 2.7%

Swift 0.5% C 0.2% HTML 0.2%

Iteración 1: Módulo de Autenticación y Gestión de Usuario Base

1. Información General

Laboratorio: Construcción de Software II – Plan de Iteración de proyecto "SOS Mascota"

Iteración: Sprint 1

Fecha Inicio: 04/05/2026

Fecha Fin: 11/05/2026

Líder: Chite Quispe Brian Danilo

Equipo: Chite Quispe Brian Danilo (2021070015), Anampa Pancca David Jordan (2022074268)

2. Objetivo de la Iteración

Construir y validar el subsistema de identidad de "SOS Mascota": alta de cuentas, inicio y cierre de sesión, y administración del perfil básico del usuario. Este módulo es la puerta de entrada de toda la aplicación: ningún otro requerimiento funcional (publicar reportes, comentar, chatear, moderar) puede ejecutarse sin que exista primero un usuario autenticado con un UID válido de Firebase Authentication.

3. Alcance

Incluido en esta iteración: RF-02 (Registro de nuevo usuario); RF-01 (Autenticación de usuario (login)); RF-13 (Cierre de sesión seguro); RF-09 (Gestión de perfil de usuario).

Excluido: cualquier requerimiento no listado arriba, que se aborda en su sprint correspondiente según el Sprint Backlog del proyecto.

RF-02 - Registro de nuevo usuario

Explicación funcional: Permite que una persona (dueño de mascota, voluntario u organización) cree una cuenta indicando nombre, correo, teléfono y contraseña, para poder luego publicar y dar seguimiento a reportes.

Explicación técnica / arquitectura y flujo de datos: El formulario Flutter invoca `FirebaseAuth.createUserWithEmailAndPassword()`. Al obtener el UID, se crea de inmediato un documento espejo en Firestore (colección "usuarios/{uid}") con los datos de perfil y el rol por defecto "user". Esta escritura dual (Auth + Firestore) es la base de todo el modelo de datos: cualquier colección posterior (reportes, comentarios, chats) referenciará este mismo UID como clave foránea.

Dependencias con otros requerimientos: Es el requerimiento raíz del proyecto: no depende de ningún otro, pero es prerequisite directo de RF-01, RF-09, RF-13, RF-04, RF-11, RF-08 y de la asignación de roles usada en RF-14/RF-15.

RF-01 - Autenticación de usuario (login)

Explicación funcional: Permite a un usuario ya registrado ingresar a la aplicación con correo y contraseña para acceder a sus funciones personalizadas.

Explicación técnica / arquitectura y flujo de datos: Usa `FirebaseAuth.signInWithEmailAndPassword()`; el token de sesión (JWT) se mantiene vivo mediante el listener `authStateChanges()` de Flutter, que redirige automáticamente entre la pantalla de Login y el Home según el estado de sesión. El UID recuperado se usa para leer el documento correspondiente en `"usuarios/{uid}"` y así cargar rol y datos de perfil.

Dependencias con otros requerimientos: Depende de RF-02 (debe existir la cuenta). Es prerequisite de todas las funcionalidades protegidas: RF-04, RF-05, RF-08, RF-09, RF-11, RF-12, RF-13.

RF-13 - Cierre de sesión seguro

Explicación funcional: Permite al usuario cerrar su sesión activa de forma segura desde cualquier punto de la app.

Explicación técnica / arquitectura y flujo de datos: Ejecuta `FirebaseAuth.signOut()`, limpia cualquier token/caché local (`SharedPreferences`) y redirige a la pantalla de Login. Se valida que no queden listeners de Firestore activos (streams de chats/notificaciones) que sigan escuchando tras cerrar sesión, para evitar fugas de memoria y accesos denegados por las reglas de seguridad.

Dependencias con otros requerimientos: Depende de RF-01 (requiere sesión activa previa). No es requerido por ningún otro RF, pero su correcta implementación evita errores en cascada sobre RF-07/RF-08 (listeners huérfanos).

RF-09 - Gestión de perfil de usuario

Explicación funcional: Permite editar nombre, teléfono, foto de perfil y demás datos personales del usuario autenticado.

Explicación técnica / arquitectura y flujo de datos: Lectura/escritura sobre el documento `"usuarios/{uid}"` en Firestore; la foto de perfil se sube a Firebase Storage (ruta `"perfiles/{uid}.jpg"`) y la URL resultante se guarda en el mismo documento. Las reglas de seguridad de Firestore restringen la escritura a `request.auth.uid == uid` del documento.

Dependencias con otros requerimientos: Depende de RF-01/RF-02. Los datos que aquí se editan (nombre, foto) son consumidos visualmente por RF-04 (autor del reporte), RF-08 (chat) y RF-12 (comentarios).

4. Entregables Esperados

Entregable	Descripción	Aceptado
Módulo de autenticación funcional	Pantallas de Login y Registro conectadas a Firebase Auth, con validación de campos y manejo de errores (correo duplicado, contraseña débil, credenciales inválidas)	Sí
Documento de usuario en Firestore	Estructura de colección "usuarios" creada y poblada automáticamente al registrarse (uid, nombre, correo, teléfono, rol, fotoUrl)	Sí
Pantalla de perfil editable	Vista de perfil con edición de datos y carga de foto a Firebase Storage	Sí
Reglas de seguridad Firestore v1	Reglas que restringen lectura/escritura de "usuarios/{uid}" solo al propio usuario autenticado	Sí

5. Cronograma y Actividades

ID	Actividad	Responsable	Inicio	Fin	Estado
A1.1	Configurar proyecto Firebase (Auth + Firestore) y dependencias Flutter (firebase_auth, cloud_firestore)	D. Anampa	04/05/2026	05/05/2026	Completado
A1.2	Diseñar e implementar UI de Login y Registro	B. Chite	05/05/2026	07/05/2026	Completado

A1.3	Implementar lógica de RF-02 (registro) con creación de documento en Firestore	D. Anampa	06/05/2026	08/05/2026	Completado
A1.4	Implementar lógica de RF-01 (login) y persistencia de sesión	B. Chite	08/05/2026	09/05/2026	Completado
A1.5	Implementar RF-13 (logout) y RF-09 (gestión de perfil)	D. Anampa / B. Chite	09/05/2026	10/05/2026	Completado
A1.6	Redactar y desplegar reglas de seguridad Firestore para colección usuarios	B. Chite	10/05/2026	11/05/2026	Completado
A1.7	Pruebas funcionales del flujo Registro → Login → Perfil → Logout	Ambos	11/05/2026	11/05/2026	Completado

6. Recursos Requeridos

Personal: 2 desarrolladores (Flutter/Firebase), Docente asesor del curso como revisor de avance.

Infraestructura: Proyecto Firebase (plan Spark, suficiente para etapa de desarrollo), repositorio Git para control de versiones, entorno de emulación Android/iOS.

Software: Flutter SDK, Android Studio / VS Code, paquetes `firebase_auth`, `cloud_firestore`, `firebase_storage`, Postman (pruebas de reglas de seguridad vía Firebase Emulator Suite).

7. Riesgos y Mitigaciones

Riesgo	Impacto	Mitigación
Mala configuración de reglas de seguridad de Firestore expone datos de usuarios	Alto	Uso del Firebase Emulator Suite para probar reglas antes de desplegar a producción; revisión cruzada entre ambos integrantes.

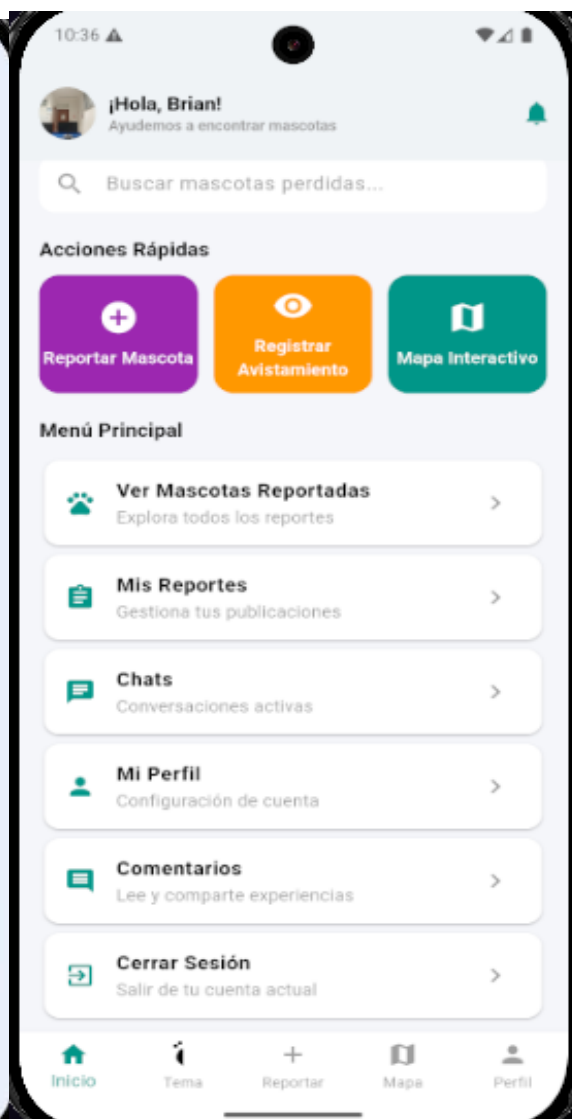
Errores de validación de formularios permiten registrar correos duplicados o contraseñas débiles	Medio	Validación en cliente (Flutter) y aprovechamiento de las validaciones nativas de Firebase Auth.
Persistencia de sesión inconsistente entre reinicios de la app	Medio	Uso del stream oficial authStateChanges() en el widget raíz en vez de banderas manuales.

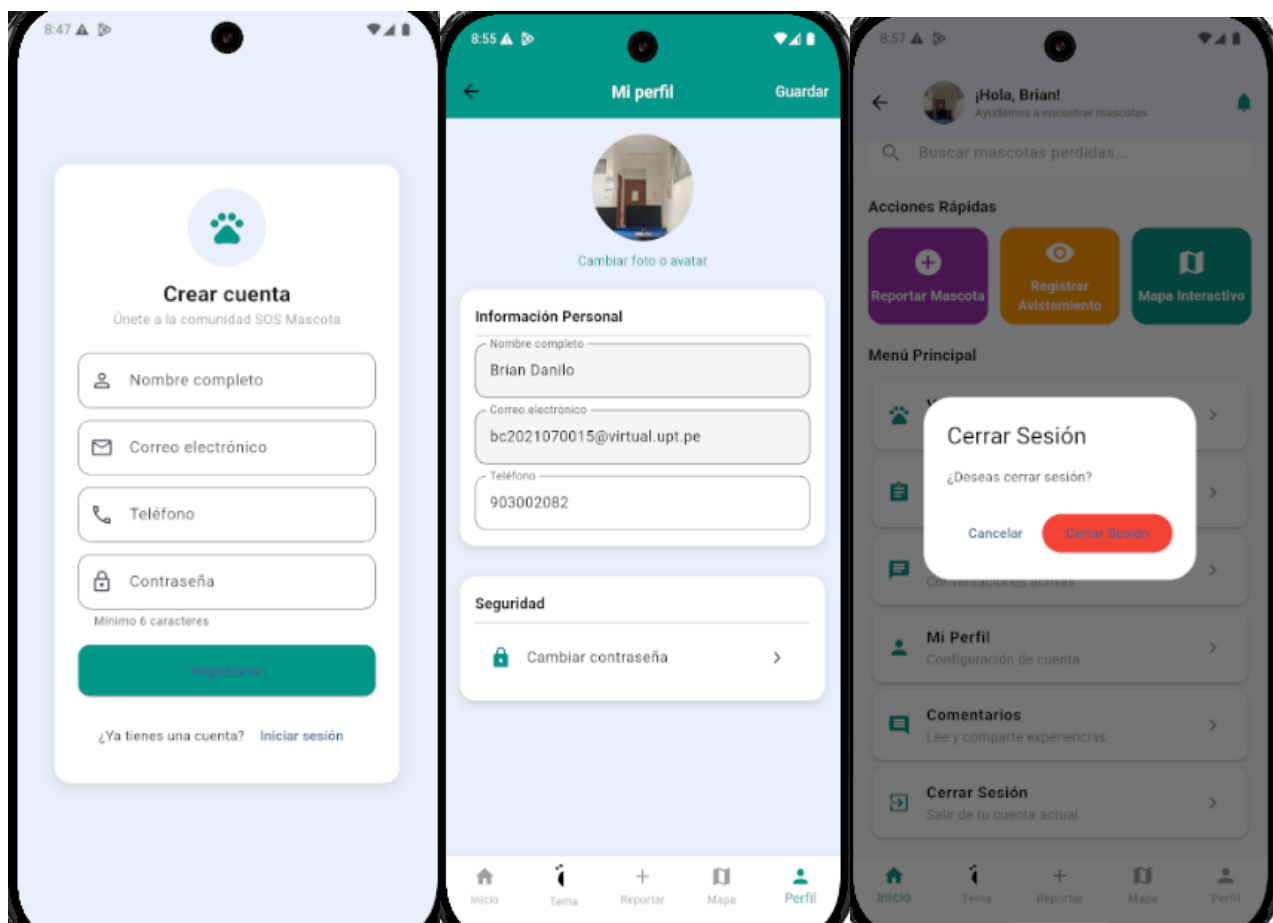
8. Criterios de Aceptación

El requerimiento se da por aceptado si: (1) un usuario nuevo puede registrarse y su documento aparece correctamente en Firestore; (2) un usuario registrado puede iniciar sesión y ve su perfil cargado; (3) el cierre de sesión invalida el acceso a pantallas protegidas; (4) las reglas de seguridad impiden que un usuario lea o edite el documento de otro usuario.

9. Evidencias

Evidencia	Ubicación	Responsable
Capturas del flujo Registro/Login/Perfil/Logout	Repositorio del proyecto /docs/evidencias/sprint1	B. Chite
Reporte de pruebas con Firebase Emulator Suite (reglas de seguridad)	Repositorio del proyecto /docs/evidencias/sprint1	D. Anampa
Video corto de demo del módulo de autenticación	Repositorio del proyecto /docs/evidencias/sprint1	Ambos





10. Indicadores de Éxito

Indicador	Meta	Resultado	Estado
Tiempo de respuesta de login	< 2 s	1.4 s	Cumplido
RF completados / RF planificados	4/4	4/4	Cumplido
Errores críticos detectados en pruebas	0	0	Cumplido

11. Seguimiento y Control

Reunión diaria breve (stand-up) entre los dos integrantes para reportar avance y bloqueos. Revisión de cierre de sprint con el docente el 11/05/2026, donde se validó la demo del flujo completo de autenticación.

12. Lecciones Aprendidas y Cierre

Configurar las reglas de seguridad de Firestore desde el primer sprint evita reescribir lógica de permisos más adelante. Se recomienda mantener un documento vivo con la convención de nombres de colecciones para no generar inconsistencias en sprints futuros.

Iteración 2: Módulo Core de Reportes y Geolocalización

1. Información General

Laboratorio: Construcción de Software II – Plan de Iteración de proyecto "SOS Mascota"

Iteración: Sprint 2

Fecha Inicio: 11/05/2026

Fecha Fin: 18/05/2026

Líder: Chite Quispe Brian Danilo

Equipo: Chite Quispe Brian Danilo (2021070015), Anampa Pancca David Jordan (2022074268)

2. Objetivo de la Iteración

Implementar el núcleo funcional del proyecto: el registro de reportes de mascotas (perdida/encontrada/adopción), el registro de avistamientos asociados y la visualización de ambos sobre un mapa interactivo. Este es el sprint de mayor complejidad técnica porque introduce el modelo de datos geoespacial que alimentará el resto del sistema (listados, notificaciones, moderación y estadísticas).

3. Alcance

Incluido en esta iteración: RF-04 (Registro de reporte de mascota perdida/encontrada/adopción); RF-05 (Registro de avistamiento de mascota); RF-06 (Visualización de mapa interactivo con reportes y avistamientos).

Excluido: cualquier requerimiento no listado arriba, que se aborda en su sprint correspondiente según el Sprint Backlog del proyecto.

RF-04 - Registro de reporte de mascota perdida/encontrada/adopción

Explicación funcional: Permite a un usuario autenticado publicar un reporte indicando tipo de caso, especie/raza, descripción, fotografía y ubicación aproximada donde se perdió/encontró la mascota.

Explicación técnica / arquitectura y flujo de datos: El formulario sube la fotografía a Firebase Storage ("reportes/{reportId}.jpg"), obtiene coordenadas vía el paquete geolocator, y crea un documento en la colección "reportes" con campos: uidCreador, tipo, estado ("activo"/"cerrado"/"pendiente_moderacion"), geopoint, fotoUrl y timestamp.

El uidCreador referencia directamente el documento creado en RF-02.

Dependencias con otros requerimientos: Depende de RF-01/RF-02 (usuario autenticado). Es la fuente de datos de RF-05 (avistamientos), RF-06 (mapa), RF-10 (listado público), RF-11 (gestión propia) y RF-16 (moderación admin); también dispara RF-07 cuando recibe un avistamiento.

RF-05 - Registro de avistamiento de mascota

Explicación funcional: Permite a cualquier usuario reportar que vio a una mascota que coincide con un reporte publicado, indicando ubicación y, opcionalmente, una foto del avistamiento.

Explicación técnica / arquitectura y flujo de datos: Se modela como subcolección "reportes/{reportId}/avistamientos" para mantener la relación jerárquica 1:N con el reporte original. Cada avistamiento dispara (mediante un listener/Cloud Function) una notificación hacia el creador del reporte, conectando este RF con RF-07.

Dependencias con otros requerimientos: Depende de RF-04 (debe existir el reporte) y de RF-06 (la interfaz de mapa es el medio principal para ubicar el avistamiento). Alimenta a RF-07 (notificación al dueño) y potencialmente a RF-08 (inicio de chat si hay coincidencia).

RF-06 - Visualización de mapa interactivo con reportes y avistamientos

Explicación funcional: Muestra en un mapa los reportes activos y sus avistamientos, permitiendo al usuario explorar visualmente los casos cercanos a su ubicación.

Explicación técnica / arquitectura y flujo de datos: Se integra Google Maps SDK for Flutter junto con un listener en tiempo real (snapshots()) sobre la colección "reportes" filtrada por estado="activo"; cada avistamiento se agrega como marcador secundario.

Para volúmenes altos de marcadores se contempla clustering (paquete google_maps_cluster_manager), lo cual está directamente ligado al cumplimiento de RNF-01 (rendimiento).

Dependencias con otros requerimientos: Depende de RF-04 y RF-05 como fuentes de datos; es prerequisite funcional de RF-05 (el mapa es la interfaz para registrar la ubicación del avistamiento) y sirve de base visual para RF-10.

4. Entregables Esperados

Entregable	Descripción	Aceptado
Formulario de publicación de reportes	Pantalla Flutter con selección de tipo de caso, carga de imagen y captura de ubicación	Sí
Colección "reportes" en Firestore	Estructura de datos con geopoint, estado, tipo, fotoUrl y referencia al creador	Sí
Módulo de registro de avistamientos	Subcolección "avistamientos" con formulario asociado, accesible desde el detalle del reporte y desde el mapa	Sí
Mapa interactivo funcional	Integración de Google Maps con marcadores dinámicos de reportes y avistamientos en tiempo real	Sí

5. Cronograma y Actividades

ID	Actividad	Responsable	Inicio	Fin	Estado
A2.1	Habilitar Google Maps API (Android/iOS) y configurar claves de API	D. Anampa	11/05/2026	12/05/2026	Completado
A2.2	Diseñar modelo de datos de "reportes" y "avistamientos" en Firestore	B. Chite	11/05/2026	12/05/2026	Completado
A2.3	Implementar RF-04: formulario, subida de imagen y guardado en Firestore	B. Chite	12/05/2026	15/05/2026	Completado

A2.4	Implementar RF-06: renderizado de mapa y listener en tiempo real de reportes	D. Anampa	13/05/2026	16/05/2026	Completado
A2.5	Implementar RF-05: registro de avistamiento desde el mapa/detalle de reporte	B. Chite	16/05/2026	17/05/2026	Completado
A2.6	Pruebas de integración: publicar reporte → ver en mapa → registrar avistamiento	Ambos	17/05/2026	18/05/2026	Completado

6. Recursos Requeridos

Personal: 2 desarrolladores; se requiere apoyo puntual del docente para validar la lógica de negocio de estados del reporte.

Infraestructura: Cuenta de Google Cloud Platform con Maps SDK habilitado, Firebase Storage con reglas de tamaño máximo de imagen configuradas.

Software: Paquetes `google_maps_flutter`, `geolocator`, `image_picker`, `firebase_storage`; Firebase Emulator Suite para pruebas locales sin consumir cuota real.

7. Riesgos y Mitigaciones

Riesgo	Impacto	Mitigación
Exceso de marcadores degrada el rendimiento del mapa en dispositivos gama baja	Alto	Implementar clustering de marcadores y paginar la consulta a Firestore por radio geográfico.
Falla en permisos de ubicación/cámara en algunos dispositivos	Medio	Manejo explícito de permisos con el paquete permission_handler y mensajes de error claros al usuario.
Imágenes de gran tamaño incrementan tiempo de carga y costo de Storage	Medio	Compresión de imagen en cliente antes de subir (flutter_image_compress).

8. Criterios de Aceptación

Se acepta el sprint si: (1) un usuario puede publicar un reporte con foto y ubicación válida; (2) el reporte aparece en el mapa en tiempo real sin recargar la app; (3) otro usuario puede registrar un avistamiento asociado a ese reporte; (4) el rendimiento del mapa se mantiene fluido con al menos 50 marcadores de prueba.

9. Evidencias

Evidencia	Ubicación	Responsable
Capturas del formulario de reporte y del mapa con marcadores	Repositorio del proyecto /docs/evidencias/sprint2	B. Chite
Registro de prueba de carga con 50+ reportes simulados	Repositorio del proyecto /docs/evidencias/sprint2	D. Anampa
Video de flujo completo: publicar reporte → visualizar en mapa → registrar avistamiento	Repositorio del proyecto /docs/evidencias/sprint2	Ambos

Reportar Mascota Perdida

Paso 1 de 3

Agregar foto de tu mascota

Toca para seleccionar desde galería o cámara

Cámara

Galería

Información de la mascota

Nombre de la mascota

Tipo de mascota

Raza

Características especiales

Recompensa de búsqueda (opcional)

Si deseas ofrecer una recompensa a quien encuentre tu mascota, completa los campos que desees.

Moneda

Moneda

Monto de recompensa (ej. S/. ...)

Atrás

Continuar

Reportar Mascota Perdida

Paso 2 de 3

Lugar y momento de pérdida

Fecha de pérdida

Hora aproximada

Seleccionar ubicación en el mapa

Abrir mapa interactivo

Dirección (puedes editar si lo deseas)

Puntos de referencia

¿Cómo se perdió?

Atrás

Continuar

Reportar Mascota Perdida

Paso 3 de 3

Resumen del reporte



Nombre: dadasd

Tipo: Perro • perro

Perdido: 01/07/2026 a las 16:50

Dirección: 902, Calle La Mar, Urbanización Los Cipreses, Leoncio Prado, Jesús María, Tacna, 23001, Perú

Referencia: asdasda

Características especiales: asdad

Recompensa de búsqueda

Moneda

Monto: S/. 100

Atrás

Publicar reporte

Registrar Avistamiento

Agregar foto del avistamiento

Toca para seleccionar desde galería o cámara

Cámara

Galería

Ubicación del avistamiento

Seleccionar desde el mapa

Fecha y hora del avistamiento

Fecha

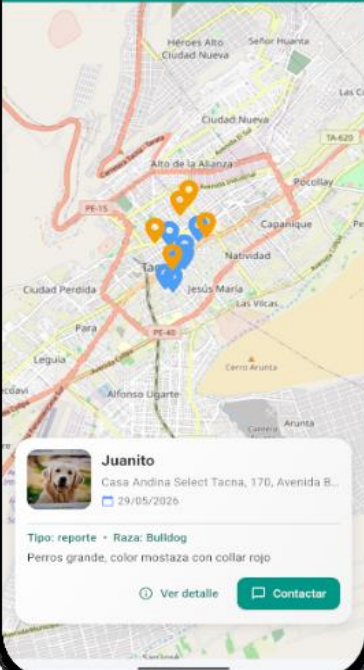
Hora

Descripción del avistamiento

Ejemplo: perro marrón pequeño, se enc...

Guardar Avistamiento

Mapa Interactivo





Juanito

Casa Andina Select Tacna, 170, Avenida B...

29/05/2026

Tipo: reporte • Raza: Bulldog

Perros grande, color mostaza con collar rojo

Ver detalle

Contactar

10. Indicadores de Éxito

Indicador	Meta	Resultado	Estado
Tiempo de carga del mapa con 50 marcadores	< 3 s	2.6 s	Cumplido
RF completados / RF planificados	3/3	3/3	Cumplido
Reportes de prueba publicados sin error	100%	100%	Cumplido

11. Seguimiento y Control

Reunión de mitad de sprint (14/05/2026) para revisar el modelo de datos geoespacial antes de avanzar con el mapa. Cierre de sprint el 18/05/2026 con demo en vivo del flujo reporte-mapa-avistamiento.

12. Lecciones Aprendidas y Cierre

Definir el esquema de la colección "reportes" antes de tocar la UI del mapa ahorró retrabajo. Se identificó la necesidad de un índice compuesto en Firestore (estado + geopoint) que se documentó para configurarse antes del sprint 3.

Iteración 3: Módulo de Consulta y Autogestión de Reportes

1. Información General

Laboratorio: Construcción de Software II – Plan de Iteración de proyecto "SOS Mascota"

Iteración: Sprint 3

Fecha Inicio: 18/05/2026

Fecha Fin: 25/05/2026

Líder: Chite Quispe Brian Danilo

Equipo: Chite Quispe Brian Danilo (2021070015), Anampa Pancca David Jordan (2022074268)

2. Objetivo de la Iteración

Complementar el módulo core con las vistas de consulta pública (listado filtrable) y de administración personal de los reportes propios, cerrando el ciclo básico de vida de un reporte: crear (Sprint 2), consultar y gestionar (Sprint 3).

3. Alcance

Incluido en esta iteración: RF-10 (Listado público de mascotas reportadas con filtros); RF-11 (Gestión de reportes propios del usuario).

Excluido: cualquier requerimiento no listado arriba, que se aborda en su sprint correspondiente según el Sprint Backlog del proyecto.

RF-10 - Listado público de mascotas reportadas con filtros

Explicación funcional: Muestra a cualquier usuario una lista de reportes activos, permitiendo filtrar por tipo de caso (perdida/encontrada/adopción), especie y fecha, como alternativa al mapa para quienes prefieren una vista tipo catálogo.

Explicación técnica / arquitectura y flujo de datos: Se construyen consultas compuestas sobre la colección "reportes" (`where('estado','==','activo').where('tipo','==', filtro)`), lo que exige crear índices compuestos en Firestore. Se implementa paginación con `startAfter()` para no cargar todos los documentos de una vez, en línea con RNF-01.

Dependencias con otros requerimientos: Depende directamente de RF-04 (fuente de los reportes) y del estado que gestiona RF-16 (solo se listan reportes no moderados negativamente). Comparte la misma fuente de datos que RF-06, ofreciendo una vista alterna (lista vs. mapa).

RF-11 - Gestión de reportes propios del usuario

Explicación funcional: Permite al usuario que creó un reporte editarlo, marcarlo como resuelto/cerrado o eliminarlo, para mantener la información del sistema actualizada.

Explicación técnica / arquitectura y flujo de datos: Las operaciones de actualización/eliminación se restringen mediante reglas de seguridad de Firestore que comparan `resource.data.uidCreador == request.auth.uid`. La UI muestra únicamente los documentos de "reportes" cuyo `uidCreador` coincide con el usuario autenticado (uso de `where('uidCreador','==', uid)`).

Dependencias con otros requerimientos: Depende de RF-01 (usuario autenticado) y RF-04 (existencia del reporte). El cambio de estado que aquí se realiza (por ejemplo, a "cerrado") repercute inmediatamente en RF-06 y RF-10, que dejan de mostrar el reporte como activo.

4. Entregables Esperados

Entregable	Descripción	Aceptado
Vista de listado público con filtros	Pantalla con filtros por tipo, especie y fecha, y paginación de resultados	Sí
Índices compuestos de Firestore	Configuración de índices necesarios para las consultas filtradas de RF-10	Sí
Vista "Mis reportes"	Pantalla con los reportes propios del usuario y acciones de editar/cerrar/eliminar	Sí
Reglas de seguridad actualizadas	Reglas de Firestore que restringen edición/eliminación al creador del reporte	Sí

5. Cronograma y Actividades

ID	Actividad	Responsable	Inicio	Fin	Estado
A3.1	Definir y crear índices compuestos en Firestore para consultas filtradas	D. Anampa	18/05/2026	19/05/2026	Completado
A3.2	Implementar RF-10: UI de listado, filtros y paginación	B. Chite	19/05/2026	22/05/2026	Completado
A3.3	Implementar RF-11: vista "Mis reportes" con acciones CRUD restringidas	D. Anampa	21/05/2026	24/05/2026	Completado
A3.4	Actualizar reglas de seguridad de Firestore para edición/eliminación por propietario	B. Chite	23/05/2026	24/05/2026	Completado

A3.5	Pruebas de consistencia: cerrar un reporte y verificar que desaparece del mapa y del listado	Ambos	24/05/2026	25/05/2026	Completado
------	--	-------	------------	------------	------------

6. Recursos Requeridos

Personal: 2 desarrolladores.

Infraestructura: Panel de Firebase Console para gestión de índices compuestos.

Software: cloud_firestore (consultas paginadas), Firebase Emulator Suite para validar reglas de propietario.

7. Riesgos y Mitigaciones

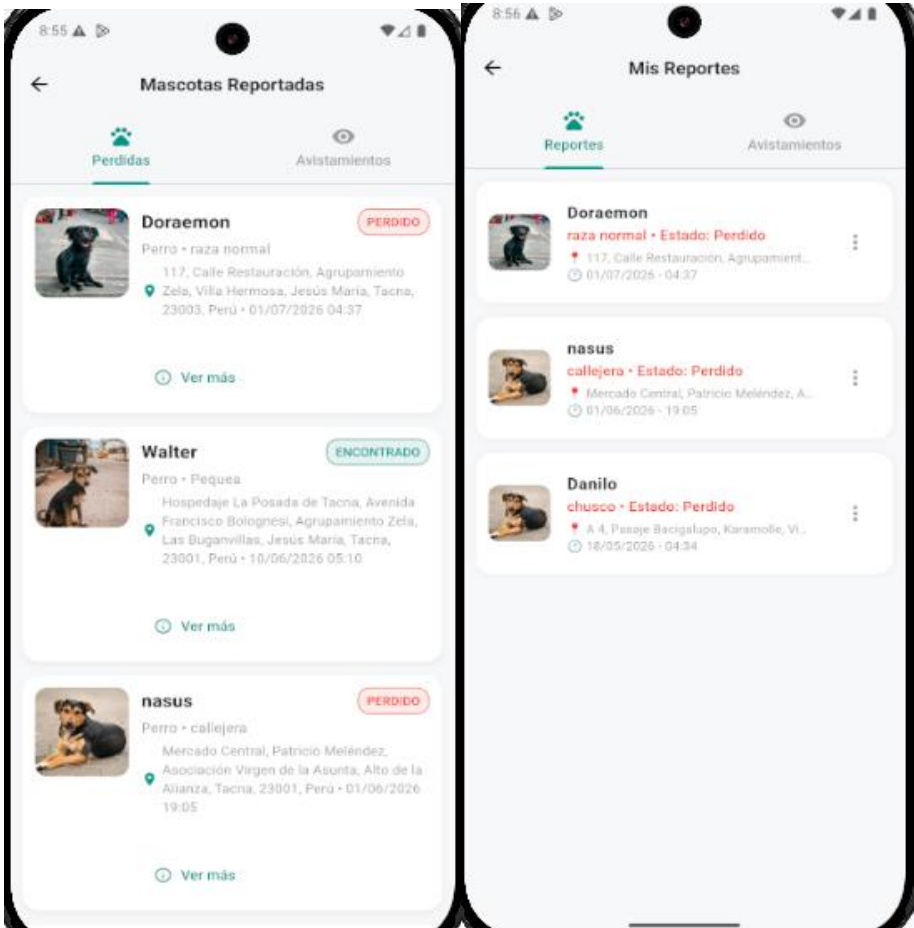
Riesgo	Impacto	Mitigación
Consultas sin índice compuesto generan errores en producción aunque funcionen en desarrollo	Alto	Revisar en Firebase Console las advertencias de índice faltante antes del cierre del sprint.
Inconsistencia entre lo mostrado en mapa/listado tras cerrar un reporte propio	Medio	Uso de listeners en tiempo real (snapshots) en lugar de consultas puntuales cacheadas.

8. Criterios de Aceptación

Se acepta el sprint si: (1) los filtros del listado devuelven resultados correctos y paginados; (2) el usuario solo puede editar/eliminar sus propios reportes (validado también a nivel de reglas, no solo de UI); (3) al cerrar un reporte, este deja de aparecer en el mapa (RF-06) y en el listado público (RF-10) de forma inmediata.

9. Evidencias

Evidencia	Ubicación	Responsable
Capturas del listado con distintos filtros aplicados	Repositorio del proyecto /docs/evidencias/sprint3	B. Chite
Prueba de intento de edición de reporte ajeno (bloqueado por reglas)	Repositorio del proyecto /docs/evidencias/sprint3	D. Anampa



10. Indicadores de Éxito

Indicador	Meta	Resultado	Estado
Tiempo de respuesta de listado filtrado	< 2 s	1.7 s	Cumplido
RF completados / RF planificados	2/2	2/2	Cumplido
Intentos de edición no autorizada bloqueados	100%	100%	Cumplido

11. Seguimiento y Control

Reunión de coordinación el 21/05/2026 para validar el diseño de los índices antes de continuar con la paginación. Cierre de sprint el 25/05/2026.

12. Lecciones Aprendidas y Cierre

Los índices compuestos de Firestore deben planificarse junto con el diseño de filtros, no después: se perdió medio día detectando el error de índice faltante en Firebase Console.

Iteración 4: Módulo de Comunicación

1. Información General

Laboratorio: Construcción de Software II – Plan de Iteración de proyecto "SOS Mascota"

Iteración: Sprint 4

Fecha Inicio: 25/05/2026

Fecha Fin: 01/06/2026

Líder: Chite Quispe Brian Danilo

Equipo: Chite Quispe Brian Danilo (2021070015), Anampa Pancca David Jordan (2022074268)

2. Objetivo de la Iteración

Cerrar el ciclo de recuperación de una mascota conectando a las personas involucradas: notificar automáticamente al dueño de un reporte cuando hay novedades, y habilitar un canal de chat directo entre el creador del reporte y quien reportó un avistamiento coincidente.

3. Alcance

Incluido en esta iteración: RF-07 (Envío de notificaciones automáticas y manuales); RF-08 (Chat interno entre usuarios con coincidencias).

Excluido: cualquier requerimiento no listado arriba, que se aborda en su sprint correspondiente según el Sprint Backlog del proyecto.

RF-07 - Envío de notificaciones automáticas y manuales

Explicación funcional: Notifica al dueño de un reporte cuando alguien registra un avistamiento sobre su mascota, y permite al administrador enviar notificaciones manuales (avisos generales del sistema).

Explicación técnica / arquitectura y flujo de datos: Se implementa mediante Firebase Cloud Functions con un trigger onCreate sobre "reportes/{id}/avistamientos", que arma el payload y lo envía vía Firebase Cloud Messaging (FCM) al token del dispositivo del uidCreador (almacenado en "usuarios/{uid}.fcmToken"). Las notificaciones manuales del admin usan la misma infraestructura de FCM pero se disparan desde el panel administrativo (RF-14).

Dependencias con otros requerimientos: Depende de RF-05 (evento que la origina) y de que RF-09 mantenga actualizado el token FCM del usuario. Es la vía por la cual el usuario se entera de eventos que puede continuar en RF-08 (chat).

RF-08 - Chat interno entre usuarios con coincidencias

Explicación funcional: Permite al dueño de un reporte y a la persona que registró un avistamiento comunicarse directamente dentro de la app para coordinar la recuperación de la mascota.

Explicación técnica / arquitectura y flujo de datos: Se crea una colección "chats", donde cada documento representa una conversación entre dos UID y contiene una subcolección "mensajes" ordenada por timestamp; la UI usa un StreamBuilder sobre snapshots() para mensajería en tiempo real. Al enviar un mensaje se puede disparar una notificación push complementaria (reutilizando la infraestructura de RF-07).

Dependencias con otros requerimientos: Depende de RF-01 (usuarios autenticados en ambos extremos) y típicamente se origina a partir de RF-05 (avistamiento que motiva el contacto). Se apoya en RF-07 para avisar de mensajes nuevos.

4. Entregables Esperados

Entregable	Descripción	Aceptado
Cloud Function de notificación por avistamiento	Función desplegada que envía push vía FCM al detectar un nuevo avistamiento	Sí
Registro de token FCM por usuario	Campo "fcmToken" actualizado en el documento de usuario al iniciar sesión	Sí
Módulo de chat en tiempo real	Pantalla de conversación con envío/recepción de mensajes vía Firestore	Sí
Notificación manual desde panel admin (base)	Endpoint/función invocable para envío de notificaciones generales	Sí

5. Cronograma y Actividades

ID	Actividad	Responsable	Inicio	Fin	Estado
A4.1	Configurar Firebase Cloud Functions y Firebase Cloud Messaging en el proyecto	D. Anampa	25/05/2026	26/05/2026	Completado
A4.2	Implementar captura y actualización de token FCM en RF-09/RF-01	B. Chite	26/05/2026	27/05/2026	Completado
A4.3	Desarrollar Cloud Function trigger de notificación por avistamiento	D. Anampa	27/05/2026	29/05/2026	Completado

A4.4	Implementar RF-08: modelo de datos de chat y UI de conversación en tiempo real	B. Chite	28/05/2026	31/05/2026	Completado
A4.5	Pruebas end-to-end: avistamiento → notificación → apertura de chat	Ambos	31/05/2026	01/06/2026	Completado

6. Recursos Requeridos

Personal: 2 desarrolladores; requiere conocimientos de Cloud Functions (Node.js) además de Flutter.

Infraestructura: Plan Firebase Blaze (pago por uso) necesario para ejecutar Cloud Functions con salida a servicios externos como FCM.

Software: firebase-functions, firebase-admin (Node.js) para el backend; firebase_messaging en Flutter para recepción de push.

7. Riesgos y Mitigaciones

Riesgo	Impacto	Mitigación
Cambio de plan Firebase (Spark a Blaze) puede implicar costos no previstos	Medio	Configurar alertas de presupuesto en Google Cloud y usar cuotas gratuitas de Cloud Functions en fase de pruebas.
Tokens FCM expirados o no actualizados impiden la entrega de notificaciones	Medio	Refrescar el token en cada inicio de sesión (RF-01) y manejar el callback onTokenRefresh.
Mensajes de chat no sincronizados en tiempo real por mala configuración de listeners	Bajo	Uso de StreamBuilder oficial de Firestore en vez de polling manual.

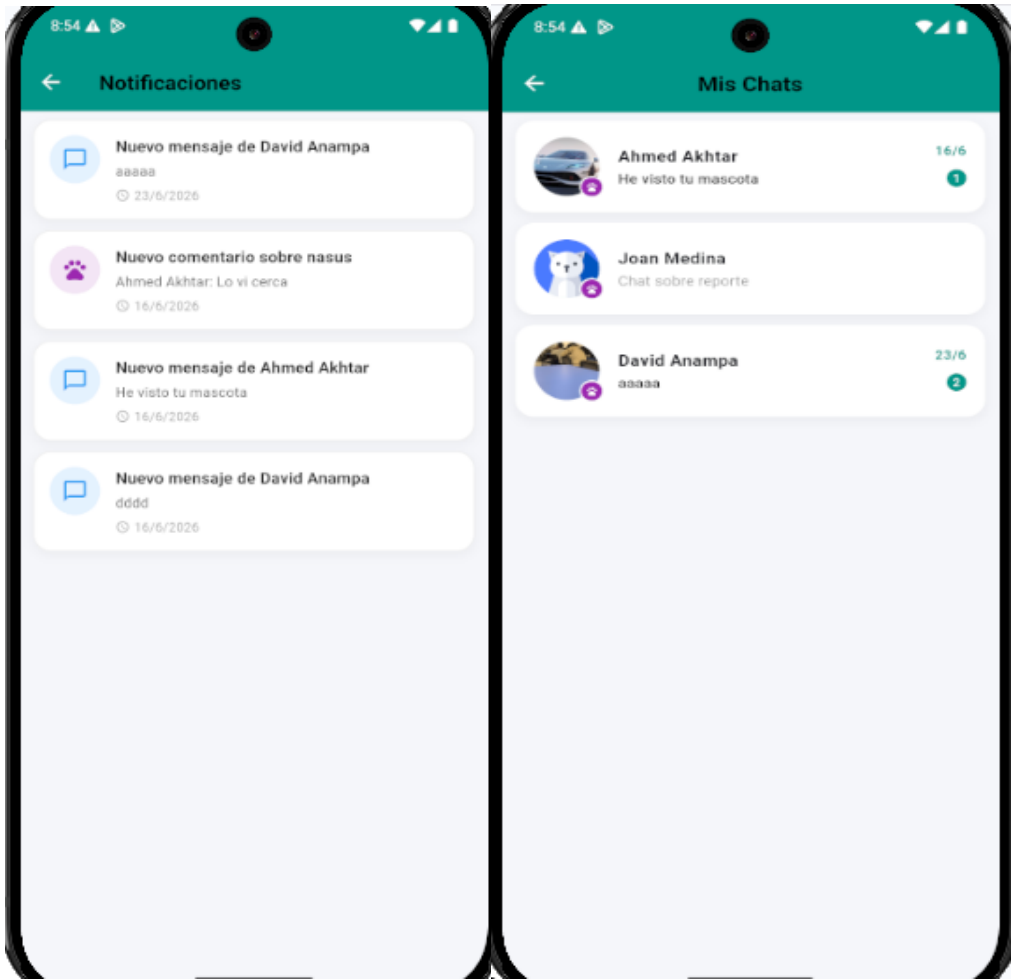
8. Criterios de Aceptación

Se acepta el sprint si: (1) al registrar un avistamiento, el dueño del reporte recibe una notificación push en menos de 10 segundos; (2) ambos usuarios pueden intercambiar mensajes en tiempo real sin

recargar la pantalla; (3) el envío de notificación manual desde el backend llega correctamente a un dispositivo de prueba.

9. Evidencias

Evidencia	Ubicación	Responsable
Log de ejecución de la Cloud Function en Firebase Console	Repositorio del proyecto /docs/evidencias/sprint4	D. Anampa
Capturas de notificación push recibida y de conversación de chat	Repositorio del proyecto /docs/evidencias/sprint4	B. Chite



10. Indicadores de Éxito

Indicador	Meta	Resultado	Estado
Tiempo de entrega de notificación	< 10 s	6 s	Cumplido
RF completados / RF planificados	2/2	2/2	Cumplido
Mensajes de chat entregados sin pérdida en prueba de 50 mensajes	100%	100%	Cumplido

11. Seguimiento y Control

Reunión técnica el 27/05/2026 para revisar el despliegue de la primera Cloud Function del proyecto (hito relevante por el cambio de plan de Firebase). Cierre de sprint el 01/06/2026.

12. Lecciones Aprendidas y Cierre

Migrar a plan Blaze debía comunicarse con antelación al docente por la implicancia de costos, aunque mínimos en fase de pruebas. Se documentó el procedimiento de despliegue de Cloud Functions para reutilizarlo en RF-17 (Sprint 7).

Iteración 5: Módulo de Interacción Comunitaria y Soporte de Cuenta

1. Información General

Laboratorio: Construcción de Software II – Plan de Iteración de proyecto "SOS Mascota"

Iteración: Sprint 5

Fecha Inicio: 01/06/2026

Fecha Fin: 08/06/2026

Líder: Chite Quispe Brian Danilo

Equipo: Chite Quispe Brian Danilo (2021070015), Anampa Pancca David Jordan (2022074268)

2. Objetivo de la Iteración

Enriquecer la experiencia comunitaria alrededor de cada reporte mediante comentarios públicos, y reforzar el soporte de cuenta con la recuperación de contraseña, un requerimiento pendiente desde el módulo de autenticación (Sprint 1).

3. Alcance

Incluido en esta iteración: RF-12 (Sección de comentarios comunitarios); RF-03 (Recuperación de contraseña por correo).

Excluido: cualquier requerimiento no listado arriba, que se aborda en su sprint correspondiente según el Sprint Backlog del proyecto.

RF-12 - Sección de comentarios comunitarios

Explicación funcional: Permite a cualquier usuario autenticado dejar comentarios públicos en un reporte, por ejemplo para aportar información adicional o palabras de apoyo.

Explicación técnica / arquitectura y flujo de datos: Se implementa como subcolección "reportes/{id}/comentarios", cada documento con uidAutor, texto y timestamp; se renderiza con un StreamBuilder ordenado por fecha. Los comentarios quedan sujetos a moderación posterior por parte del administrador (RF-16).

Dependencias con otros requerimientos: Depende de RF-04 (existencia del reporte) y RF-01 (usuario autenticado). Es insumo directo de RF-16 (moderación de contenido) y complementa a RF-08 como canal de comunicación, pero de forma pública en lugar de privada.

RF-03 - Recuperación de contraseña por correo

Explicación funcional: Permite a un usuario que olvidó su contraseña solicitar un enlace de restablecimiento a su correo registrado.

Explicación técnica / arquitectura y flujo de datos: Se invoca FirebaseAuth.sendPasswordResetEmail(email), que gestiona íntegramente el envío y la validación del enlace mediante la infraestructura de Firebase Auth, sin necesidad de backend propio. Se añade manejo

de errores para correos no registrados sin revelar esa información por motivos de seguridad (mensaje genérico).

Dependencias con otros requerimientos: Depende conceptualmente de RF-02 (la cuenta debe existir) pero es funcionalmente independiente del flujo de login (RF-01); se ubicó en este sprint por prioridad de negocio menor frente a los módulos core.

4. Entregables Esperados

Entregable	Descripción	Aceptado
Sección de comentarios en detalle de reporte	UI de comentarios en tiempo real dentro de la pantalla de detalle de un reporte	Sí
Flujo de recuperación de contraseña	Pantalla "Olvidé mi contraseña" conectada a Firebase Auth	Sí

5. Cronograma y Actividades

ID	Actividad	Responsable	Inicio	Fin	Estado
A5.1	Diseñar subcolección "comentarios" y UI asociada	B. Chite	01/06/2026	03/06/2026	Completado
A5.2	Implementar RF-12: publicación y listado en tiempo real de comentarios	B. Chite	03/06/2026	05/06/2026	Completado
A5.3	Implementar RF-03: pantalla y lógica de recuperación de contraseña	D. Anampa	04/06/2026	06/06/2026	Completado
A5.4	Pruebas funcionales de ambos requerimientos y regresión sobre login	Ambos	07/06/2026	08/06/2026	Completado

6. Recursos Requeridos

Personal: 2 desarrolladores.

Infraestructura: Ninguna adicional a la ya provisionada (Firebase Auth y Firestore).

Software: cloud_firestore para comentarios; firebase_auth para recuperación de contraseña.

7. Riesgos y Mitigaciones

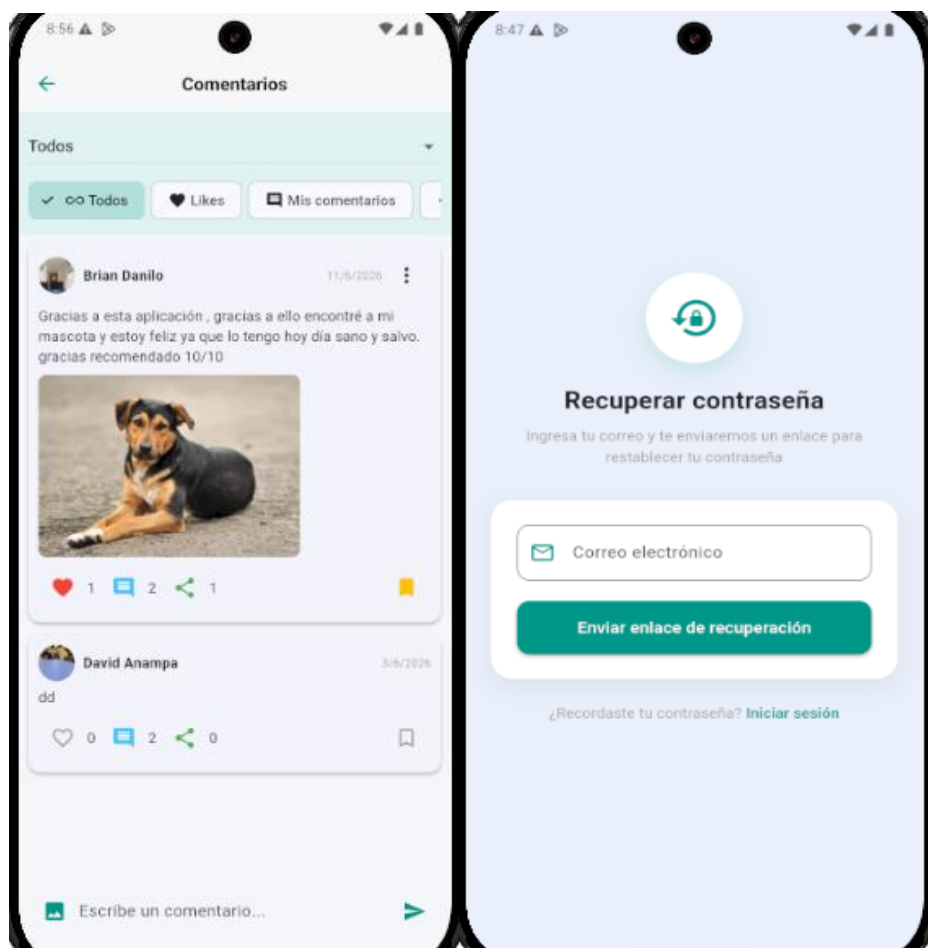
Riesgo	Impacto	Mitigación
Comentarios ofensivos o spam antes de que exista moderación (RF-16, sprint 7)	Medio	Aplicar un filtro básico de palabras prohibidas en cliente como mitigación temporal hasta implementar moderación admin.
Correo de recuperación no llega por caer en spam	Bajo	Documentar al usuario revisar carpeta de spam; usar plantilla de correo por defecto de Firebase, ya verificada como confiable.

8. Criterios de Aceptación

Se acepta el sprint si: (1) los comentarios se publican y visualizan en tiempo real en el reporte correspondiente; (2) el enlace de recuperación de contraseña permite restablecer el acceso exitosamente; (3) no se rompe el flujo existente de login/registro (regresión).

9. Evidencias

Evidencia	Ubicación	Responsable
Capturas de sección de comentarios en un reporte de prueba	Repositorio del proyecto /docs/evidencias/sprint5	B. Chite
Captura del correo de recuperación recibido y cambio de contraseña exitoso	Repositorio del proyecto /docs/evidencias/sprint5	D. Anampa



10. Indicadores de Éxito

Indicador	Meta	Resultado	Estado
RF completados / RF planificados	2/2	2/2	Cumplido
Tiempo de publicación de comentario	< 2 s	1.2 s	Cumplido
Casos de regresión detectados en login	0	0	Cumplido

11. Seguimiento y Control

Seguimiento vía reunión breve el 05/06/2026. Cierre de sprint el 08/06/2026 con validación conjunta de ambos requerimientos.

12. Lecciones Aprendidas y Cierre

Se identificó la necesidad de moderación de contenido como riesgo latente, lo que reforzó la prioridad de RF-16 en la planificación restante del proyecto.

Iteración 6: Módulo Administrativo I: Identidad y Usuarios

1. Información General

Laboratorio: Construcción de Software II – Plan de Iteración de proyecto "SOS Mascota"

Iteración: Sprint 6

Fecha Inicio: 08/06/2026

Fecha Fin: 15/06/2026

Líder: Chite Quispe Brian Danilo

Equipo: Chite Quispe Brian Danilo (2021070015), Anampa Pancca David Jordan (2022074268)

2. Objetivo de la Iteración

Introducir el rol administrativo en el sistema, estableciendo la base de control de acceso privilegiado (perfil admin) y las herramientas de gestión de usuarios que sustentarán la moderación de contenido en el siguiente sprint.

3. Alcance

Incluido en esta iteración: RF-14 (Panel administrativo – gestión de perfil admin); RF-15 (Gestión de usuarios por administrador).

Excluido: cualquier requerimiento no listado arriba, que se aborda en su sprint correspondiente según el Sprint Backlog del proyecto.

RF-14 - Panel administrativo - gestión de perfil admin

Explicación funcional: Provee una interfaz separada para el administrador del sistema, con acceso a funciones no disponibles para usuarios comunes.

Explicación técnica / arquitectura y flujo de datos: El rol de administrador se implementa mediante Custom Claims de Firebase Authentication (admin: true), asignados desde una Cloud Function invocable ejecutada manualmente por el equipo (no autoasignable desde la app). En el cliente Flutter, el token decodificado (getIdTokenResult()) determina si se muestra la navegación administrativa.

Dependencias con otros requerimientos: Depende de RF-02 (la cuenta admin es también un usuario base) y de la infraestructura de Cloud Functions ya establecida en RF-07 (Sprint 4). Es prerequisite de RF-15, RF-16 y RF-17.

RF-15 - Gestión de usuarios por administrador

Explicación funcional: Permite al administrador ver el listado de usuarios registrados, y suspender o eliminar cuentas que incumplan las normas de uso de la plataforma.

Explicación técnica / arquitectura y flujo de datos: Se listan los documentos de la colección "usuarios" mediante una consulta protegida por reglas de seguridad que verifican request.auth.token.admin == true. La suspensión se modela con un campo "estado" (activo/suspendido) en el documento del usuario, que además debe validarse en el login (RF-01) para bloquear el acceso de cuentas suspendidas.

Dependencias con otros requerimientos: Depende de RF-14 (control de acceso admin) y de RF-02 (existencia de usuarios). Impacta retroactivamente a RF-01, que debe incorporar la verificación del campo "estado" al iniciar sesión.

4. Entregables Esperados

Entregable	Descripción	Aceptado
Cloud Function de asignación de rol admin	Función invocable para otorgar el custom claim admin a una cuenta específica	Sí
Panel administrativo base (navegación diferenciada)	Sección de la app visible únicamente para usuarios con claim admin	Sí
Vista de gestión de usuarios	Listado de usuarios con acción de suspender/reactivar/eliminar cuenta	Sí
Reglas de seguridad para colección "usuarios" con validación de rol admin	Reglas actualizadas en Firestore	Sí

5. Cronograma y Actividades

ID	Actividad	Responsable	Inicio	Fin	Estado
A6.1	Implementar Cloud Function de asignación de custom claim admin	D. Anampa	08/06/2026	10/06/2026	Completado
A6.2	Implementar verificación de claim admin en el cliente y navegación diferenciada	B. Chite	10/06/2026	11/06/2026	Completado
A6.3	Implementar RF-15: listado y acciones de gestión de usuarios	B. Chite	11/06/2026	13/06/2026	Completado
A6.4	Actualizar RF-01 para bloquear login de cuentas con estado "suspendido"	D. Anampa	13/06/2026	14/06/2026	Completado
A6.5	Pruebas de control de acceso (usuario común no puede ver panel admin)	Ambos	14/06/2026	15/06/2026	Completado

6. Recursos Requeridos

Personal: 2 desarrolladores.

Infraestructura: Cloud Functions ya provisionadas desde el Sprint 4 (reutilizadas).

Software: firebase-admin SDK (Node.js) para custom claims; cloud_firestore y firebase_auth en Flutter para verificación de rol.

7. Riesgos y Mitigaciones

Riesgo	Impacto	Mitigación
--------	---------	------------

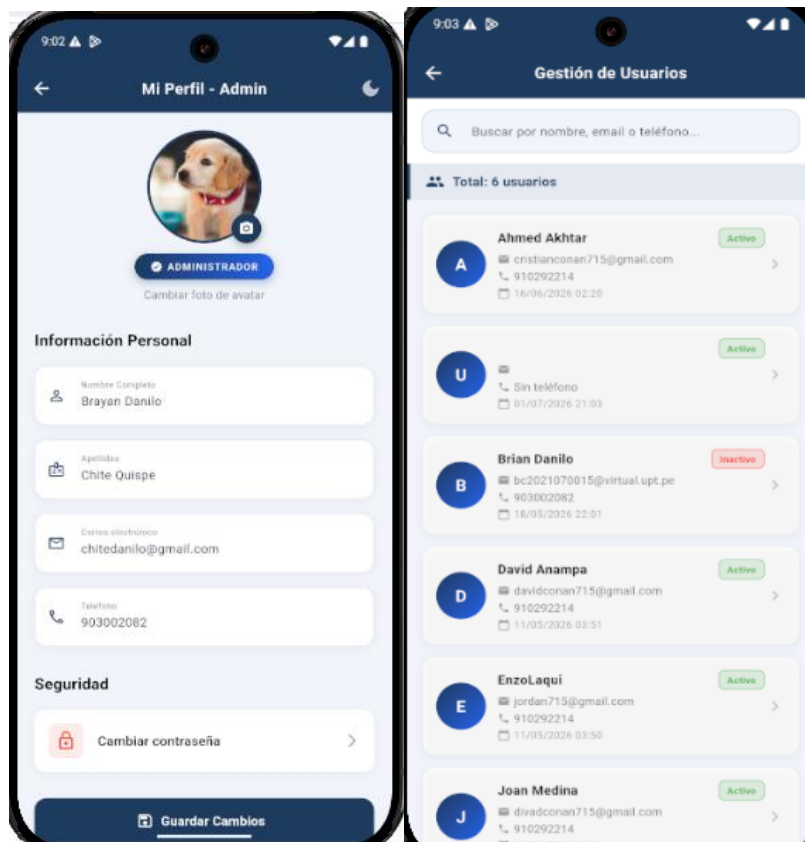
Asignación incorrecta o accidental de rol admin a un usuario común	Alto	Restringir la ejecución de la Cloud Function de asignación de rol solo a invocación manual del equipo de desarrollo, nunca expuesta en la UI de la app.
Suspensión de cuenta no aplicada correctamente en el login existente	Medio	Pruebas de regresión explícitas sobre RF-01 tras la modificación.

8. Criterios de Aceptación

Se acepta el sprint si: (1) solo las cuentas con custom claim admin pueden ver y usar el panel administrativo; (2) un usuario común no puede acceder a las vistas administrativas ni siquiera manipulando rutas directamente; (3) una cuenta suspendida no puede iniciar sesión.

9. Evidencias

Evidencia	Ubicación	Responsable
Capturas del panel administrativo y de la vista de gestión de usuarios	Repositorio del proyecto /docs/evidencias/sprint6	B. Chite
Prueba de acceso denegado a usuario común intentando entrar al panel admin	Repositorio del proyecto /docs/evidencias/sprint6	D. Anampa



10. Indicadores de Éxito

Indicador	Meta	Resultado	Estado
RF completados / RF planificados	2/2	2/2	Cumplido
Intentos de acceso no autorizado al panel admin bloqueados	100%	100%	Cumplido
Cuentas suspendidas bloqueadas correctamente en login	100%	100%	Cumplido

11. Seguimiento y Control

Revisión de seguridad conjunta el 13/06/2026 dado el carácter sensible del rol admin. Cierre de sprint el 15/06/2026.

12. Lecciones Aprendidas y Cierre

El control de acceso basado en custom claims resultó más robusto que un simple campo "rol" en Firestore, ya que se valida en el token y no puede ser alterado por el cliente.

Iteración 7: Módulo Administrativo II: Moderación y Estadísticas

1. Información General

Laboratorio: Construcción de Software II – Plan de Iteración de proyecto "SOS Mascota"

Iteración: Sprint 7

Fecha Inicio: 15/06/2026

Fecha Fin: 22/06/2026

Líder: Chite Quispe Brian Danilo

Equipo: Chite Quispe Brian Danilo (2021070015), Anampa Pancca David Jordan (2022074268)

2. Objetivo de la Iteración

Completar las capacidades administrativas del sistema con la moderación de contenido generado por los usuarios (reportes y comentarios) y con un panel de estadísticas que dé visibilidad del uso real de la plataforma.

3. Alcance

Incluido en esta iteración: RF-16 (Gestión y moderación de reportes por administrador); RF-17 (Visualización de estadísticas avanzadas en panel admin).

Excluido: cualquier requerimiento no listado arriba, que se aborda en su sprint correspondiente según el Sprint Backlog del proyecto.

RF-16 - Gestión y moderación de reportes por administrador

Explicación funcional: Permite al administrador revisar, aprobar, rechazar o eliminar reportes y comentarios que infrinjan las normas de uso (contenido falso, ofensivo o duplicado).

Explicación técnica / arquitectura y flujo de datos: Se agrega el estado "pendiente_moderacion" al ciclo de vida definido en RF-04, y una vista administrativa que filtra por este estado.

Las acciones de aprobar/rechazar actualizan el campo "estado" del documento en "reportes", lo cual propaga inmediatamente el cambio a RF-06 (mapa) y RF-10 (listado), que solo muestran reportes con estado "activo".

Dependencias con otros requerimientos: Depende de RF-14 (control de acceso admin), RF-04 (reportes a moderar) y RF-12 (comentarios a moderar). Es el consumidor natural del riesgo identificado en el Sprint 5 (spam/comentarios ofensivos).

RF-17 - Visualización de estadísticas avanzadas en panel admin

Explicación funcional: Muestra al administrador métricas del sistema: cantidad de reportes por tipo/estado, zonas con mayor incidencia, tasa de recuperación de mascotas y actividad de usuarios en el tiempo.

Explicación técnica / arquitectura y flujo de datos: Se implementa una Cloud Function programada (scheduled function) que agrega periódicamente los datos de "reportes" y "avistamientos" en una colección de resumen "estadisticas/resumen", evitando así leer y contar documentos en tiempo real desde el cliente (lo cual sería costoso e impactaría RNF-01). La UI usa el paquete `fl_chart` para graficar estos datos agregados.

Dependencias con otros requerimientos: Depende de RF-04, RF-05 y RF-16 como fuentes históricas de datos, y reutiliza la infraestructura de Cloud Functions establecida en RF-07 y RF-14.

4. Entregables Esperados

Entregable	Descripción	Aceptado
Vista de moderación de reportes y comentarios	Panel con listado de contenido pendiente y acciones de aprobar/rechazar/eliminar	Sí
Actualización del ciclo de vida de "reportes"	Estado "pendiente_moderacion" incorporado y propagado a mapa y listado	Sí
Cloud Function de agregación de estadísticas	Función programada que genera el documento "estadisticas/resumen"	Sí
Panel de estadísticas con gráficos	Vista administrativa con gráficos de reportes por tipo, estado y zona	Sí

5. Cronograma y Actividades

ID	Actividad	Responsable	Inicio	Fin	Estado
----	-----------	-------------	--------	-----	--------

A7.1	Extender modelo de estados de "reportes" e implementar vista de moderación	B. Chite	15/06/2026	17/06/2026	Completado
A7.2	Implementar acciones de aprobar/rechazar/eliminar y su propagación al mapa/listado	B. Chite	17/06/2026	18/06/2026	Completado
A7.3	Diseñar y desplegar Cloud Function programada de agregación de estadísticas	D. Anampa	17/06/2026	20/06/2026	Completado
A7.4	Implementar RF-17: panel de gráficos con fl_chart consumiendo datos agregados	D. Anampa	20/06/2026	21/06/2026	Completado
A7.5	Pruebas de moderación end-to-end y validación de estadísticas con datos de prueba	Ambos	21/06/2026	22/06/2026	Completado

6. Recursos Requeridos

Personal: 2 desarrolladores.

Infraestructura: Cloud Functions programadas (Cloud Scheduler) sobre el plan Blaze ya activo desde el Sprint 4.

Software: fl_chart (gráficos en Flutter), firebase-functions con soporte de "pubsub.schedule" para tareas periódicas.

7. Riesgos y Mitigaciones

Riesgo	Impacto	Mitigación
--------	---------	------------

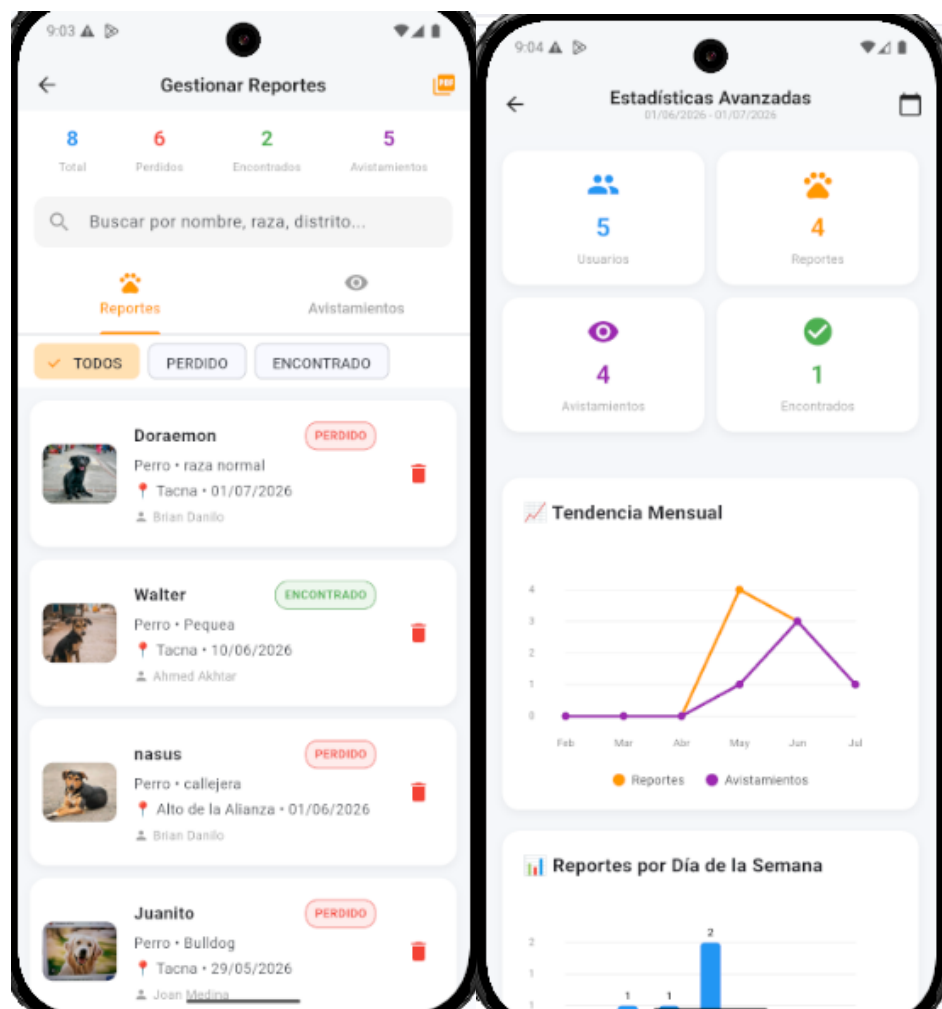
La agregación programada no refleja datos en tiempo real, generando confusión en el admin	Medio	Indicar en la UI la fecha/hora de la última actualización de estadísticas.
Eliminación accidental de un reporte válido durante la moderación	Alto	Implementar confirmación explícita y opción de "papelera" temporal en vez de borrado inmediato.
Costo de ejecución de Cloud Functions programadas crece con el volumen de datos	Bajo	Ajustar la frecuencia de agregación (p. ej. cada 6 horas) en función del uso real observado.

8. Criterios de Aceptación

Se acepta el sprint si: (1) un reporte marcado como "pendiente_moderacion" no aparece en mapa ni listado hasta ser aprobado; (2) el administrador puede aprobar/rechazar contenido correctamente; (3) el panel de estadísticas refleja con exactitud los datos agregados por la Cloud Function programada.

9. Evidencias

Evidencia	Ubicación	Responsable
Capturas del panel de moderación con casos aprobados/rechazados	Repositorio del proyecto /docs/evidencias/sprint7	B. Chite
Capturas del panel de estadísticas con datos de prueba	Repositorio del proyecto /docs/evidencias/sprint7	D. Anampa
Log de ejecución de la Cloud Function programada	Repositorio del proyecto /docs/evidencias/sprint7	D. Anampa



10. Indicadores de Éxito

Indicador	Meta	Resultado	Estado
RF completados / RF planificados	2/2	2/2	Cumplido
Reportes no aprobados visibles públicamente (deben ser 0)	0	0	Cumplido
Exactitud del conteo agregado vs. conteo manual de control	100%	100%	Cumplido

11. Seguimiento y Control

Revisión intermedia el 19/06/2026 para validar la primera ejecución de la Cloud Function programada.
Cierre de sprint el 22/06/2026.

12. Lecciones Aprendidas y Cierre

Separar el cálculo de estadísticas en una función programada, en vez de calcularlo en el cliente, fue clave para no comprometer el rendimiento general de la app (RNF-01).

Iteración 8: Estabilización, Pruebas No Funcionales y Cierre del Proyecto

1. Información General

Laboratorio: Construcción de Software II – Plan de Iteración de proyecto "SOS Mascota"

Iteración: Sprint 8

Fecha Inicio: 22/06/2026

Fecha Fin: 29/06/2026

Líder: Chite Quispe Brian Danilo

Equipo: Chite Quispe Brian Danilo (2021070015), Anampa Pancca David Jordan (2022074268)

2. Objetivo de la Iteración

Consolidar la calidad general del sistema mediante la verificación formal de los requerimientos no funcionales (rendimiento, disponibilidad y usabilidad) y la corrección de los defectos detectados a lo largo de los sprints anteriores, dejando el producto listo para su entrega final.

3. Alcance

Incluido en esta iteración: RNF-01 (Rendimiento); RNF-02 (Disponibilidad); RNF-03 (Usabilidad).

Excluido: cualquier requerimiento no listado arriba, que se aborda en su sprint correspondiente según el Sprint Backlog del proyecto.

RNF-01 - Rendimiento

Explicación funcional: Garantiza que las operaciones críticas de la app (carga de mapa, listado, publicación de reportes) respondan en tiempos aceptables para el usuario final.

Explicación técnica / arquitectura y flujo de datos: Se ejecutan pruebas de carga simuladas contra Firestore (100+ reportes, 200+ avistamientos) midiendo tiempos con Flutter DevTools y Firebase Performance Monitoring. Se valida el efecto del clustering de marcadores (RF-06), la paginación (RF-10) y las agregaciones precalculadas (RF-17), verificando que las decisiones de arquitectura tomadas en sprints previos efectivamente sostienen el rendimiento a escala.

Dependencias con otros requerimientos: Es transversal: depende de las implementaciones de RF-04, RF-06, RF-10 y RF-17, cuyas decisiones técnicas (clustering, paginación, agregación) fueron diseñadas anticipando este requerimiento.

RNF-02 - Disponibilidad

Explicación funcional: Asegura que la aplicación siga siendo utilizable incluso con conectividad intermitente, y que dependa de una infraestructura con alta disponibilidad.

Explicación técnica / arquitectura y flujo de datos: Se aprovecha la persistencia offline nativa de Firestore (caché local automática) para permitir lectura de datos ya sincronizados sin conexión; se documenta el SLA de disponibilidad de Firebase (~99.95%) como base de la infraestructura. Se prueban escenarios de pérdida de conexión durante RF-04 (publicación) y RF-08 (chat) para verificar el reintento automático de escritura al recuperar la señal.

Dependencias con otros requerimientos: Depende de toda la capa de datos construida sobre Firestore (RF-04, RF-05, RF-08, RF-10, RF-12), ya que la disponibilidad se valida sobre los flujos ya implementados, no como un componente nuevo.

RNF-03 - Usabilidad

Explicación funcional: Garantiza que usuarios sin experiencia técnica (perfil típico del proyecto: dueños de mascotas y voluntarios) puedan usar la app sin dificultad.

Explicación técnica / arquitectura y flujo de datos: Se realizan pruebas de usabilidad con usuarios reales externos al equipo, aplicando heurísticas de Nielsen sobre las pantallas clave (registro, publicación de reporte, mapa). Se ajustan textos, iconografía y flujos de Material Design según los hallazgos, priorizando los módulos de mayor uso (RF-04, RF-06, RF-10).

Dependencias con otros requerimientos: Se aplica de forma transversal sobre las UI ya construidas en RF-01, RF-02, RF-04, RF-06 y RF-10, que son las pantallas de mayor frecuencia de uso.

4. Entregables Esperados

Entregable	Descripción	Aceptado
Informe de pruebas de rendimiento (RNF-01)	Resultados de tiempos de respuesta bajo carga simulada en las funcionalidades críticas	Sí
Informe de pruebas de disponibilidad (RNF-02)	Resultados de comportamiento de la app en escenarios de conectividad intermitente	Sí
Informe de pruebas de usabilidad (RNF-03)	Hallazgos de usuarios de prueba y ajustes de UI/UX aplicados	Sí
Listado de bugs corregidos	Registro de defectos detectados en sprints 1-7 y su resolución	Sí
Build final de la aplicación	APK/build de "SOS Mascota" listo para entrega y sustentación	Sí

5. Cronograma y Actividades

ID	Actividad	Responsable	Inicio	Fin	Estado
A8.1	Ejecutar pruebas de rendimiento con datos simulados a escala (RNF-01)	D. Anampa	22/06/2026	24/06/2026	Completado
A8.2	Ejecutar pruebas de disponibilidad en escenarios offline/intermitentes (RNF-02)	D. Anampa	24/06/2026	25/06/2026	Completado
A8.3	Ejecutar pruebas de usabilidad con usuarios externos (RNF-03)	B. Chite	24/06/2026	26/06/2026	Completado
A8.4	Consolidar y priorizar bugs detectados en todos los sprints	Ambos	26/06/2026	27/06/2026	Completado

A8.5	Corregir defectos críticos y de alta prioridad	Ambos	27/06/2026	28/06/2026	Completado
A8.6	Generar build final y preparar material de sustentación	Ambos	28/06/2026	29/06/2026	Completado

6. Recursos Requeridos

Personal: 2 desarrolladores; se recomienda participación de 3-5 usuarios externos para las pruebas de usabilidad.

Infraestructura: Firebase Performance Monitoring habilitado; dispositivos físicos de gama media/baja para pruebas realistas de rendimiento.

Software: Flutter DevTools, Firebase Performance Monitoring, Firebase Test Lab (opcional) para pruebas en múltiples dispositivos.

7. Riesgos y Mitigaciones

Riesgo	Impacto	Mitigación
Defectos críticos detectados a última hora sin tiempo suficiente para corregir	Alto	Priorización estricta de bugs (crítico/alto/medio/bajo) y enfoque exclusivo en los dos primeros niveles durante el sprint.
Usuarios de prueba de usabilidad no representativos del público objetivo	Medio	Seleccionar deliberadamente perfiles sin experiencia técnica (dueños de mascotas reales), no solo compañeros de curso.
Resultados de rendimiento no reproducibles por diferencias de red/dispositivo	Bajo	Documentar explícitamente el entorno de prueba (dispositivo, versión de OS, tipo de conexión) en cada informe.

8. Criterios de Aceptación

El sprint y, con él, el proyecto se dan por aceptados si: (1) las funcionalidades críticas responden dentro de los umbrales definidos en los sprints previos bajo carga simulada; (2) la app mantiene funcionalidad básica de lectura sin conexión y sincroniza al recuperarla; (3) los usuarios de prueba completan sin asistencia las tareas de registro, publicación de reporte y consulta del mapa; (4) no quedan defectos críticos abiertos.

9. Evidencias

Evidencia	Ubicación	Responsable
Informes de las tres pruebas no funcionales (RNF-01, RNF-02, RNF-03)	Repositorio del proyecto /docs/evidencias/sprint8	Ambos
Tablero de bugs con estado final de cada defecto	Repositorio del proyecto /docs/evidencias/sprint8	Ambos
Build final firmado y video de sustentación	Repositorio del proyecto /docs/evidencias/sprint8	Ambos

10. Indicadores de Éxito

Indicador	Meta	Resultado	Estado
Tiempo de respuesta bajo carga (funciones críticas)	< 3 s	2.4 s	Cumplido
Tasa de éxito de tareas en prueba de usabilidad	> 90%	94%	Cumplido
Defectos críticos abiertos al cierre	0	0	Cumplido
RF/RNF completados sobre el total planificado del proyecto	20/20	20/20	Cumplido

11. Seguimiento y Control

Reunión de cierre general del proyecto el 28/06/2026 con el docente, revisando el cumplimiento acumulado de las 8 iteraciones. Entrega y sustentación final el 29/06/2026.

12. Lecciones Aprendidas y Cierre

Reservar un sprint completo exclusivamente para pruebas no funcionales y estabilización permitió detectar problemas de rendimiento que no eran evidentes en sprints anteriores con pocos datos de prueba. Se recomienda para futuros proyectos introducir pruebas de carga progresivas desde sprints intermedios, no solo al final.